

Хитрый лис
Спецификация интеграции фронтенда с backend
Версия документа: 2026-04-28

Назначение

Этот документ нужен фронтендеру как рабочая спецификация для интеграции с backend. Он фиксирует:

- какие сценарии должны поддерживаться на клиенте;
- какой HTTP и WebSocket контракт считать целевым;
- какие поля состояния являются источником истины;
- как обрабатывать reconnect, ошибки и рассинхрон;
- какие части backend уже соответствуют этому контракту, а какие еще надо довести.

Важно

1. Это не описание "как сейчас случайно сериализуется Go-структура", а целевой публичный контракт между frontend и backend.
2. Фронтенд не должен строить правила игры самостоятельно. Источник истины всегда backend.
3. Фронтенд не должен опираться на скрытые серверные поля вроде culprit_id.
4. При расхождении между локальным UI и серверным state побеждает state от backend.

1. Общая модель взаимодействия

Архитектурная модель для клиента:

- HTTP используется для входа, создания игры, входа в лобби, старта игры и первичной загрузки snapshot.
- WebSocket используется для live-обновлений внутри комнаты и для игровых команд. Подключение к WebSocket допускается только после того, как сервер подтвердил участие пользователя в игре.
- PostgreSQL на backend является единственным источником истины по игре.
- Комната WebSocket на сервере хранит только активные соединения и не хранит саму игру.

Минимальный сценарий MVP, который frontend должен уметь пройти:

1. Игрок получает токен.
2. Игрок создает игру или входит в существующую.
3. Только после успешного create/join клиент имеет право открывать WebSocket на /ws/games/{id}.
4. Игрок открывает экран lobby и видит участников.
5. После старта игры клиент получает snapshot.
6. Активный игрок отправляет игровую команду.
7. Сервер применяет команду, сохраняет state и отправляет update всем.
8. Если соединение рвется, клиент переподключается и получает актуальный snapshot заново.

Главный принцип для UI:

- backend говорит, "что есть";
- frontend решает только, "как это показать".

Практическое правило:

- не открывать websocket по gameId сразу после выбора комнаты из списка;
- websocket открывается только после успешного ответа на POST create/join;
- если открыть websocket раньше, сервер вернет forbidden и соединение будет закрыто.

2. Статус backend на 2026-04-28

=====
Что уже близко к целевому контракту:

- доменная логика игры;
- набор игровых команд:
 choose_goal, roll_auto, take_clue, reveal_suspects, end_turn, accuse;
- базовый WebSocket handler для /ws/games/{id};
- формат update/error ответов по WebSocket;
- сервис применения команд с транзакцией;
- HTTP endpoints для create/join/start/get;
- базовая работа с lobby snapshot.

Что еще не завершено и должно считаться "планируемым контрактом":

- финальная auth-схема;
- полная фильтрация публичного state;
- окончательная live-presence модель лобби;
- финальный контракт lobby_update/game_update, если будет разделение типов сообщений;
- серверный таймер хода и авто-доигрывание хода после дедлайна.

Следствие для frontend:

- экранную и state-логику уже можно проектировать по этому документу;
- мок-сервер и адаптер API лучше писать сразу по этому контракту;
- не надо привязываться к случайным именам полей из сырого domain state.

=====
3. Роли экранов на frontend
=====

Экран auth:

- получает токен или guest identity;
- сохраняет токен локально;
- переводит пользователя в lobby.

Экран lobby:

- создает игру;
- позволяет войти в игру по id;
- показывает список игроков;
- показывает, можно ли стартовать игру;
- открывает websocket комнаты только после успешного create/join;
- показывает live-изменения состава комнаты, если websocket уже открыт;
- переводит пользователя на экран game после старта.

Экран game:

- получает initial snapshot;
- использует уже открытый websocket или переоткрывает его на game screen;
- показывает state игры;
- отправляет игровые команды;
- обрабатывает update/error;
- переживает reconnect без потери консистентности.

=====
4. Аутентификация и идентичность пользователя
=====

Целевое поведение:

- frontend хранит токен доступа;
- каждый HTTP запрос передает токен в Authorization header;
- WebSocket передает токен либо в Authorization header, либо query param token;
- сервер извлекает из токена user_id;

- user_id связывается с участником игры в таблице game_players.

Рекомендуемый формат на клиенте:

- HTTP: Authorization: Bearer <token>
- WS: /ws/games/{gameId}?token=<token>

Требования к фронтенду:

- не пытаться вычислять user_id самостоятельно;
- держать токен в одном центральном месте;
- при 401 очищать сессию и возвращать пользователя на auth screen;
- не открывать websocket без токена;
- не открывать websocket до успешного create/join.

5. HTTP контракт

Ниже описан рабочий контракт, под который frontend стоит строить API layer.

5.1. Создать игру

POST /api/v1/games

Request body:

```
{
  "title": "Комната Ромы",
  "visibility": "public"
}
```

Допустимые значения visibility:

- public
- private

Response 201:

```
{
  "game": {
    "id": "g1",
    "status": "waiting"
  },
  "player": {
    "user_id": "u1",
    "seat": 0
  },
  "title": "Комната Ромы",
  "visibility": "public",
  "joinCode": null
}
```

Что делает frontend после успеха:

- сохраняет game id в router state;
- переходит на lobby screen для этой игры;
- может сразу открыть websocket комнаты;
- делает GET состояния lobby как initial snapshot или fallback.

5.2. Войти в игру

POST /api/v1/games/{id}/join

Response 200:

```
{
  "game": {
    "id": "g1",
    "status": "waiting"
  },

```

```
"player": {
  "user_id": "u2",
  "seat": 1
}
}
```

Ошибки:

- 404, если игра не найдена;
- 409, если игра уже заполнена;
- 409, если игра уже стартовала;
- 403, если пользователь не имеет доступа.

Что делает frontend после успеха:

- переходит на lobby screen;
- только после 200 OK открывает websocket комнаты;
- не пытается открывать websocket заранее.

5.3. Войти в приватную игру по коду

POST /api/v1/games/join-by-code

Request body:

```
{
  "code": "482193"
}
```

Response 200:

```
{
  "game": {
    "id": "g1",
    "status": "waiting"
  },
  "player": {
    "user_id": "u2",
    "seat": 1
  }
}
```

Ошибки:

- 400, если код невалидный;
- 404, если игра по коду не найдена;
- 409, если игра заполнена или уже стартовала.

5.4. Получить lobby snapshot

GET /api/v1/games/{id}

Response 200:

```
{
  "game": {
    "id": "g1",
    "title": "Комната Ромы",
    "status": "waiting",
    "visibility": "public",
    "host_username": "roma",
    "players": [
      {
        "user_id": "u1",
        "seat": 0,
        "display_name": "Player 1",
        "is_me": true
      },
      {
        "user_id": "u2",
        "seat": 1,

```

```

    "display_name": "Player 2",
    "is_me": false
  }
],
"can_start": true,
"min_players": 2,
"max_players": 4
}
}

```

Что важно для UI:

- can_start приходит с сервера;
- frontend не должен сам решать, достаточно ли игроков;
- seat отображается как порядок за столом;
- host_username приходит с сервера и отображается как владелец комнаты.

5.5. Выйти из комнаты

POST /api/v1/games/{id}/leave

Response 200:

```

{
  "game_deleted": false,
  "new_host_user_id": "u2",
  "new_host_username": "player2"
}

```

Семантика:

- если после выхода игроков не осталось, комната удаляется;
- если хост вышел, лидерство передается другому игроку;
- если комната не пустая, websocket-участники получают обновленный lobby state.

5.6. Старт игры

POST /api/v1/games/{id}/start

Response 200:

```

{
  "game": {
    "id": "g1",
    "status": "active"
  },
  "redirect": {
    "route": "/game/g1"
  }
}

```

После успеха frontend:

- делает GET snapshot игры или принимает первый update по websocket;
- использует уже открытый websocket;
- переходит на game screen.

5.7. Получить игровой snapshot

GET /api/v1/games/{id}/state

Response 200:

```

{
  "state": { ...PublicGameState... }
}

```

Этот endpoint нужен в трех случаях:

- initial load экрана game;

- ручное восстановление после ошибки;
- fallback, если websocket еще не открыт или сломан.

6. WebSocket контракт

6.1. Подключение

URL:

/ws/games/{gameId}?token=<token>

Подключение допускается только если:

- пользователь аутентифицирован;
- пользователь уже является участником игры;
- для создателя комнаты это выполняется сразу после create;
- для второго игрока это выполняется только после успешного join/join-by-code.

Что происходит сразу после подключения:

1. Сервер проверяет доступ.
2. Сервер добавляет соединение в room.
3. Сервер отправляет initial state.
4. Дальше соединение работает в режиме command -> update/error.

Важно:

- если открыть websocket до join, сервер вернет forbidden и соединение будет закрыто;
- websocket не должен открываться "на всякий случай" до того, как backend добавил пользователя в game_players.

6.2. Общий формат клиентского сообщения

```
{
  "id": "req-123",
  "type": "command",
  "command": "choose_goal",
  "payload": {
    "goal": "clue"
  }
}
```

Поля:

- id: client-generated request id;
- type: всегда command;
- command: имя игровой команды;
- payload: параметры команды.

Frontend обязан:

- генерировать уникальный id на каждую команду;
- держать локальную связь "какая кнопка отправила этот request";
- не переиспользовать один и тот же id для разных команд.

6.3. Формат update от сервера

Базовый вариант для game screen:

```
{
  "id": "req-123",
  "type": "game_update",
  "payload": {
    "state": { ...PublicGameState... },
    "events": [
      {
```

```

    "type": "goal_chosen",
    "data": {
      "goal": "clue"
    }
  }
]
}
}

```

Допустимый переходный вариант MVP:

```

{
  "id": "req-123",
  "type": "update",
  "payload": {
    "state": { ...PublicGameState... },
    "events": [...]
  }
}

```

Для lobby live-update допускается отдельный тип:

```

{
  "id": "",
  "type": "lobby_update",
  "payload": {
    "game": { ...LobbySnapshot... }
  }
}

```

Семантика:

- id указывает, какая клиентская команда привела к этому state;
- если update пришел не в ответ на локальную команду, id может быть пустым;
- state всегда важнее events;
- events нужны для анимации и лога действий;
- frontend должен быть готов к тому, что lobby и game update могут иметь разные payload.

6.4. Формат error от сервера

```

{
  "id": "req-123",
  "type": "error",
  "payload": {
    "code": "not_your_turn",
    "message": "It is not your turn."
  }
}

```

Правило обработки:

- ошибка не должна ломать websocket session;
- ошибка показывается пользователю в UI;
- после error frontend не меняет state локально;
- если UI был оптимистичным, он обязан откатиться к последнему серверному snapshot.

7. Игровые команды, которые должен уметь отправлять frontend

7.1. choose_goal

Когда доступна:

- только активному игроку;
- когда phase == choose_goal.

Request:

```
{
  "id": "req-1",
  "type": "command",
  "command": "choose_goal",
  "payload": {
    "goal": "clue"
  }
}
```

Допустимые значения goal:

- clue
- suspect

7.2. roll_auto

Когда доступна:

- только активному игроку;
- когда phase == rolling.

Request:

```
{
  "id": "req-2",
  "type": "command",
  "command": "roll_auto",
  "payload": {}
}
```

Frontend не бросает кубики сам. Он только инициирует серверный бросок и потом отрисовывает результат из events/state.

7.3. take_clue

Когда доступна:

- только активному игроку;
- когда phase == action;
- когда pending == clue.

7.4. reveal_suspects

Когда доступна:

- только активному игроку;
- когда phase == action;
- когда pending == suspect.

7.5. end_turn

Когда доступна:

- когда серверный state позволяет завершить ход;
- frontend не должен сам придумывать исключения.

Для MVP можно ориентироваться на:

- phase == end_turn;
- или явный флаг can_end_turn в публичном state, если backend его добавит.

7.6. accuse

Когда доступна:

- только когда backend помечает обвинение допустимым;
- frontend не должен раскрывать подозреваемого, если сервер этого еще не разрешил.

Request:

```
{
  "id": "req-3",
  "type": "command",
  "command": "accuse",
  "payload": {
    "suspectId": 4
  }
}
```

Важно:

- suspectId в payload должен относиться к публичному идентификатору подозреваемого;
- frontend не должен отправлять индекс массива, если backend договорится об отдельном публичном id.

8. Публичное состояние игры: PublicGameState

Frontend должен работать не с внутренним server state, а с его публичной проекцией. Рекомендуемый объект:

```
{
  "id": "g1",
  "status": "active",
  "phase": "choose_goal",
  "turn": 3,
  "version": 7,
  "active_seat": 1,
  "turn_deadline_at": "2026-04-28T19:45:00Z",
  "me": {
    "user_id": "u2",
    "seat": 1,
    "is_active": true
  },
  "players": [
    {
      "user_id": "u1",
      "seat": 0,
      "display_name": "Player 1",
      "position": 0
    },
    {
      "user_id": "u2",
      "seat": 1,
      "display_name": "Player 2",
      "position": 0
    }
  ],
  "goal": {
    "selected": true,
    "type": "clue"
  },
  "pending": "none",
  "fox_track": 3,
  "fox_escape_at": 15,
  "clues_found": 2,
  "clues_total": 12,
  "suspects": [
    {
      "id": 0,
      "revealed": true,
      "excluded": false
    }
  ],
  "result": "none",
}
```

```

"actions": {
  "can_choose_goal": false,
  "can_roll": true,
  "can_take_clue": false,
  "can_reveal_suspects": false,
  "can_end_turn": false,
  "can_accuse": false
}
}

```

Пояснения по полям:

- id: идентификатор игры;
- status: waiting | active | finished;
- phase: choose_goal | rolling | action | end_turn;
- turn: номер хода;
- version: версия состояния, растёт при каждом изменении;
- active_seat: чей сейчас ход;
- turn_deadline_at: серверный дедлайн текущего хода;
- me: данные текущего пользователя внутри игры;
- players: публичные данные всех игроков;
- goal: цель текущего хода;
- pending: какое действие надо завершить после успешного броска;
- fox_track: позиция лиса;
- fox_escape_at: порог поражения;
- clues_found / clues_total: прогресс по подсказкам;
- suspects: публичный список подозреваемых;
- result: none | win | lose;
- actions: серверный набор разрешенных кнопок.

Почему нужен actions block:

- чтобы frontend не вычислял правила из нескольких косвенных полей;
- чтобы UI был проще;
- чтобы backend мог менять правила без переписывания клиента.

9. Что frontend не должен получать от backend

Никогда не отдавать клиенту в обычном игровом state:

- culprit_id до конца игры;
- внутренние технические поля репозитория;
- приватные флаги, по которым можно вычислить скрытую информацию;
- "сырой" JSON из базы без публичной фильтрации.

Если в текущей реализации backend случайно это уже сериализует, frontend не должен на это опираться. Нужно считать это временным багом, а не частью контракта.

10. Правила работы frontend с state

Основной принцип:

- любой пришедший server state полностью заменяет локальный актуальный state.

Рекомендуемый порядок:

1. При GET /state сохранить snapshot как source of truth.
2. После успешного create/join открыть websocket комнаты.
3. Пока websocket устанавливается, продолжать показывать последний snapshot.
4. При каждом update заменять локальный state новым state из payload.
5. Events писать в отдельный короткий лог для анимации и истории хода.
6. При reconnect не пытаться "доставлять" пропущенные события. Просто принять новый snapshot.

Что не надо делать:

- не открывать websocket до create/join;
- не мутировать локальный state вручную после нажатия кнопки;
- не предсказывать, что сервер "скорее всего" ответит;
- не пытаться синхронизировать состояние частичными патчами без необходимости.

Оптимистичный UI:

- для MVP лучше не использовать;
- допустим только для состояния "кнопка отправлена / pending";
- итоговая модель игры должна обновляться только с сервера.

11. Матрица экранов и действий

11.1. Auth screen

UI должен уметь:

- ввести или получить токен;
- сохранить токен;
- проверить, что токен валиден через любой доступный auth endpoint;
- перейти в lobby.

11.2. Lobby screen

Lobby должен уметь:

- создать игру;
- войти в игру;
- войти в приватную игру по коду;
- показать список участников;
- показать, кто я;
- показать статус waiting/active/finished;
- показать host_username;
- показать кнопку start только если сервер разрешает;
- открыть websocket только после успешного create/join;
- обновляться от websocket, если сервер шлет lobby_update;
- перейти в game, когда игра стартовала.

11.3. Game screen

Game screen должен уметь:

- загрузить snapshot;
- использовать websocket комнаты;
- показать статус соединения: connecting / online / reconnecting / offline;
- показать чей ход;
- показать текущую phase;
- показать таймер хода по turn_deadline_at;
- показать разрешенные действия;
- отправлять команды;
- показать server errors;
- переживать page refresh и reconnect.

12. Обработка ошибок

Ошибки делятся на три группы.

12.1. HTTP ошибки

Примеры:

- 401 unauthorized;

- 403 forbidden;
- 404 game not found;
- 409 game full / invalid state transition;
- 500 internal error.

Поведение frontend:

- 401: сброс сессии и возврат на auth;
- 403: показать "доступ запрещен";
- 404: показать "игра не найдена";
- 409: показать понятное бизнес-сообщение;
- 500: показать общий серверный сбой и дать retry.

12.2. WebSocket error messages

Backend уже мапит как минимум такие коды:

- not_your_turn
- invalid_phase
- game_not_active
- game_finished
- goal_already_set
- goal_not_set
- no_pending_action
- pending_not_clue
- pending_not_suspect
- all_clues_collected
- suspect_not_revealed
- suspect_excluded
- forbidden
- internal_error

Поведение frontend:

- показывать короткое уведомление;
- не закрывать сокет;
- не менять игровой state;
- оставлять пользователю возможность продолжать игру.

12.3. Ошибки транспорта

Примеры:

- сокет закрылся;
- сервер отклонил websocket как forbidden, если игрок еще не joined;
- таймаут сети;
- браузер ушел offline;
- backend перезапустился.

Поведение frontend:

- показать reconnecting status;
- начать повторные подключения с backoff;
- после удачного подключения запросить или принять свежий snapshot;
- скрыть статус reconnecting только после получения актуального state;
- если forbidden получен до join, не крутить reconnect бесконечно, а сначала выполнить join.

13. Reconnect и повторное подключение

Целевой алгоритм:

1. WebSocket закрылся.
2. UI помечает соединение как reconnecting.
3. Клиент делает повторное подключение с backoff, например 1s, 2s, 5s.
4. После подключения сервер отправляет полный update со state.

5. Клиент заменяет локальный state этим snapshot.
6. UI снова становится interactive.

Frontend не должен:

- пытаться воспроизводить локально команды, которые были "в полете";
- считать, что последний локальный state все еще истинен;
- восстанавливать очередь событий самостоятельно.

Если пользователь нажал кнопку и сразу произошел disconnect:

- команда считается неподтвержденной;
- после reconnect пользователь видит фактический state от сервера;
- если команда не применена, пользователь может отправить ее заново вручную.

Если reconnect происходит на экране комнаты:

- сначала убедиться, что пользователь все еще участник комнаты;
- затем переоткрыть websocket;
- если сервер вернул forbidden, вернуться к повторному join-flow или закрыть экран комнаты.

14. Версии состояния и request correlation

Зачем frontend нужны version и request id:

- version помогает понять, что state действительно изменился;
- request id связывает server response с конкретным пользовательским действием;
- это упрощает логирование, дебаг и отображение loading на кнопках.

Рекомендуемая логика:

- когда кнопка отправляет команду, сохранить pendingRequestId;
- когда приходит update с этим id, снять loading;
- когда приходит error с этим id, снять loading и показать ошибку;
- если пришел update с большей version без моего id, просто принять его как новое состояние игры.

15. Рекомендации по store на frontend

Рекомендуемая схема store:

- auth store: token, session status;
- lobby store: текущая игра в стадии waiting;
- game store: currentState, connectionStatus, pendingRequestIds, eventLog;
- api/ws adapters: отдельный слой, не смешивать с компонентами.

Компоненты должны:

- подписываться на store;
- диспатчить intent-экшены вроде createGame, joinGame, joinByCode, connectGameSocket, sendChooseGoal;
- не содержать в себе сетевую логику напрямую.

16. Минимальный чеклист для frontend implementation

До интеграции с живым backend:

- подготовить типы для HTTP contract;
- подготовить типы для WebSocket request/update/error;
- описать PublicGameState и LobbySnapshot;
- сделать mock API;

- сделать mock WebSocket server или локальный event simulator;
- реализовать lobby flow;
- реализовать game flow;
- реализовать reconnect state machine.

Во время интеграции:

- проверить, что websocket открывается только после create/join;
- проверить, что имена команд совпадают с backend;
- проверить, что error codes совпадают с backend;
- проверить, что state не содержит скрытых полей;
- проверить, что reconnect действительно возвращает консистентный snapshot.

После интеграции:

- добавить client-side логирование request id и version;
- проверить поведение на двух вкладках/двух игроках;
- проверить page refresh в середине партии;
- проверить закрытие сокета во время чужого хода и своего хода;
- проверить кейс "создатель комнаты подключается сразу, второй игрок только после join".

17. Definition of Done для фронтендера

Frontend-часть считается готовой, если:

- пользователь может пройти auth и попасть в lobby;
- пользователь может создать или открыть игру;
- lobby корректно показывает игроков и возможность старта;
- websocket комнаты открывается только после того, как backend подтвердил участие пользователя в игре;
- game screen показывает initial snapshot;
- game screen открывает websocket и получает live updates;
- активный игрок может отправлять все доступные команды;
- неактивный игрок видит обновления без возможности ломать ход;
- server errors отображаются без рассинхрона UI;
- reconnect не ломает игровое состояние;
- клиент не использует скрытые серверные поля для логики.

18. Короткая памятка для разработчика frontend

- Доверяй серверу, не локальной догадке.
- Рендери из state, а не из предположений.
- WebSocket открывай только после create/join.
- Команды отправляй только по websocket.
- Snapshot получай по HTTP и при reconnect.
- Не считай внутренний state backend публичным контрактом.
- Не используй culprit_id на клиенте.
- Держи version и request id в store.
- Любой update от сервера может изменить весь экран.

Конец документа.